Lab_Assignment_15.2 B37

AI Assisted Coding 2303A52077

Task Descriptor-1: (Setup Fast API Backend) Install fastapi

```
PS C:\Users\gauta\OneDrive\Documents\SEM-6\AI\docs\codes> py -m pip install fastapi uvicorn
Collecting fastapi
  Downloading fastapi-0.135.1-py3-none-any.whl.metadata (30 kB)
Collecting uvicorn
  Downloading uvicorn-0.41.0-py3-none-any.whl.metadata (6.7 kB)
Collecting starlette>=0.46.0 (from fastapi)
  Downloading starlette-0.52.1-py3-none-any.whl.metadata (6.3 kB)
Collecting pydantic>=2.7.0 (from fastapi)
  Downloading pydantic-2.12.5-py3-none-any.whl.metadata (90 kB)
Collecting typing-extensions>=4.8.0 (from fastapi)
  Downloading typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting typing-inspection>=0.4.2 (from fastapi)
  Downloading typing_inspection-0.4.2-py3-none-any.whl.metadata (2.6 kB)
Collecting annotated-doc>=0.0.2 (from fastapi)
  Downloading annotated_doc-0.0.4-py3-none-any.whl.metadata (6.6 kB)
Collecting click>=7.0 (from uvicorn)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting h11>=0.8 (from uvicorn)
  Downloading h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Collecting colorama (from click>=7.0->uvicorn)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting annotated-types>=0.6.0 (from pydantic>=2.7.0->fastapi)
  Downloading annotated_types-0.7.0-py3-none-any.whl.metadata (15 kB)
Collecting pydantic-core==2.41.5 (from pydantic>=2.7.0->fastapi)
  Downloading pydantic_core-2.41.5-cp313-cp313-win_amd64.whl.metadata (7.4 kB)
Collecting anyio<5,>=3.6.2 (from starlette>=0.46.0->fastapi)
  Downloading anyio-4.12.1-py3-none-any.whl.metadata (4.3 kB)
Collecting idna>=2.8 (from anyio<5,>=3.6.2->starlette>=0.46.0->fastapi)
  Downloading idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Downloading fastapi-0.135.1-py3-none-any.whl (116 kB)
Downloading uvicorn-0.41.0-py3-none-any.whl (68 kB)
Downloading annotated_doc-0.0.4-py3-none-any.whl (5.3 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)
Downloading h11-0.16.0-py3-none-any.whl (37 kB)
Downloading pydantic-2.12.5-py3-none-any.whl (463 kB)
Downloading pydantic_core-2.41.5-cp313-cp313-win_amd64.whl (2.0 MB)
   ---------------------------------------- 2.0/2.0 MB 1.3 MB/s  0:00:01
```

```
Downloading pydantic_core-2.41.5-cp313-cp313-win_amd64.whl (2.0 MB)
   ---------------------------------------- 2.0/2.0 MB 1.3 MB/s  0:00:01
Downloading annotated_types-0.7.0-py3-none-any.whl (13 kB)
Downloading starlette-0.52.1-py3-none-any.whl (74 kB)
Downloading anyio-4.12.1-py3-none-any.whl (113 kB)
Downloading idna-3.11-py3-none-any.whl (71 kB)
Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Downloading typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: typing-extensions, idna, h11, colorama, annotated-types, annotated-doc, typing-inspection,
pydantic-core, click, anyio, uvicorn, starlette, pydantic, fastapi
   ---------------------------------------- 10/14 [uvicorn]  WARNING: The script uvicorn.exe is installed in 'C:\Users\gau
ta\AppData\Local\Programs\Python\Python313\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
   ---------------------------------------- 13/14 [fastapi]  WARNING: The script fastapi.exe is installed in 'C:\Users\gau
ta\AppData\Local\Programs\Python\Python313\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed annotated-doc-0.0.4 annotated-types-0.7.0 anyio-4.12.1 click-8.3.1 colorama-0.4.6 fastapi-0.135.1 h
11-0.16.0 idna-3.11 pydantic-2.12.5 pydantic-core-2.41.5 starlette-0.52.1 typing-extensions-4.15.0 typing-inspection-0.4.2
 uvicorn-0.41.0
```

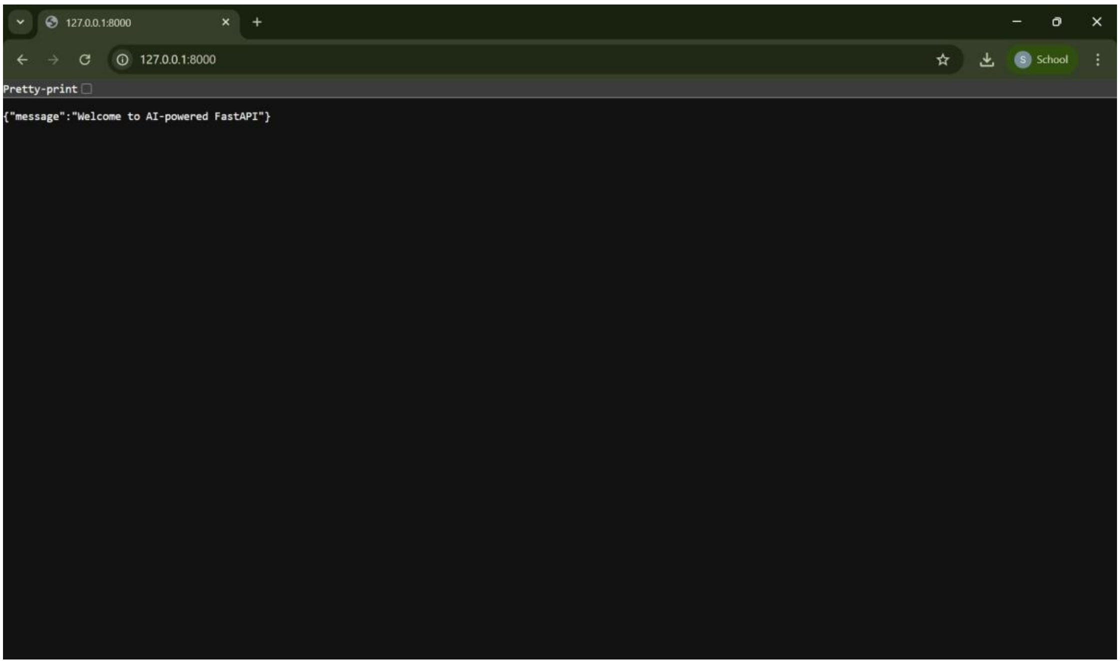
Code

ai 15.2 > main.py > read_root

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
```

Output

```
INFO:     Will watch for changes in these directories: ['C:\\Users\\gauta\\OneD
 15.2']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [1868] using StatReload
INFO:     Started server process [19872]
INFO:     Waiting for application startup.
```



Explanation

In this task, I set up a basic FastAPI backend application. First, I installed the required packages, FastAPI and Uvicorn, using Python. Then, I created a Python file called main.py and wrote a simple FastAPI program with a basic API endpoint (/). After that, I ran the server using Uvicorn in the terminal. Finally, I opened the URL http://127.0.0.1:8000/ in the browser and checked that the API returned the message "Welcome to AI-powered FastAPI." This confirmed that the backend server was working well.

Task Descriptor- 2: (API – Create and Read Items) Execution

```
INFO:     Will watch for changes in these directories: ['C:\\Users\\gauta
 15.2']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [23564] using StatReload
INFO:     Started server process [20464]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

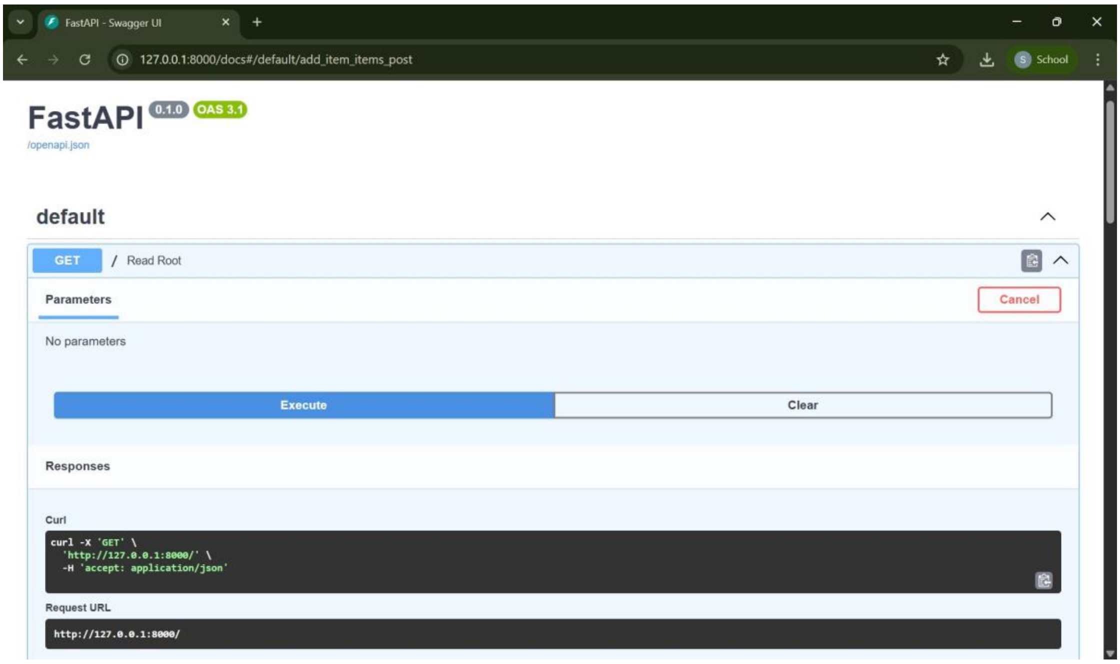
Code

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
from fastapi import FastAPI

app = FastAPI()

# Temporary storage
items = []

# Root endpoint
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}

# Read items
@app.get("/items")
def get_items():
    return items

# Create item
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

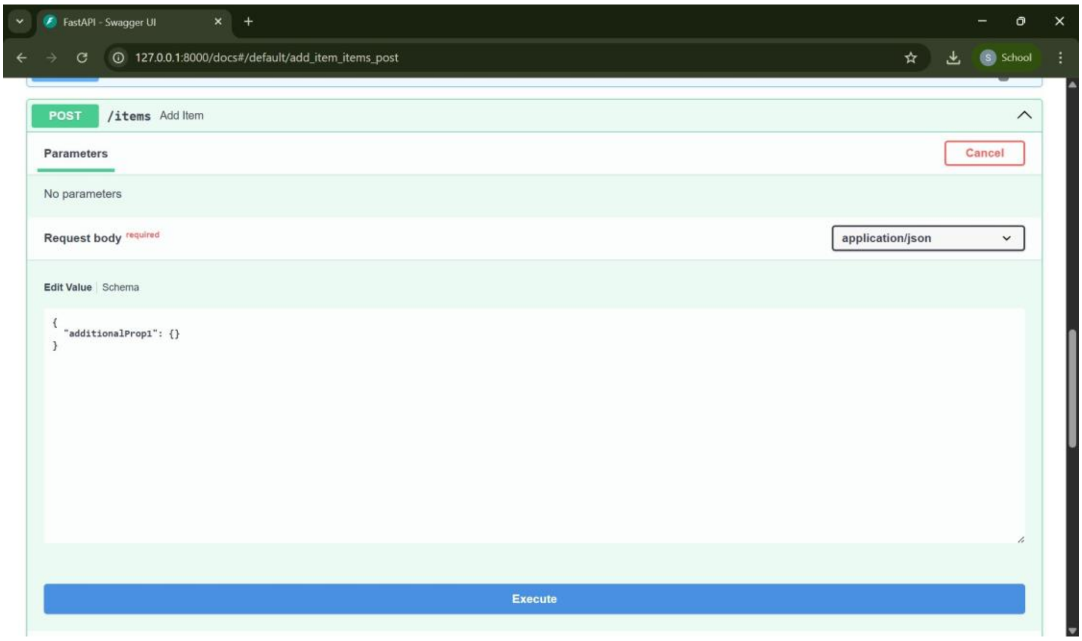
Output

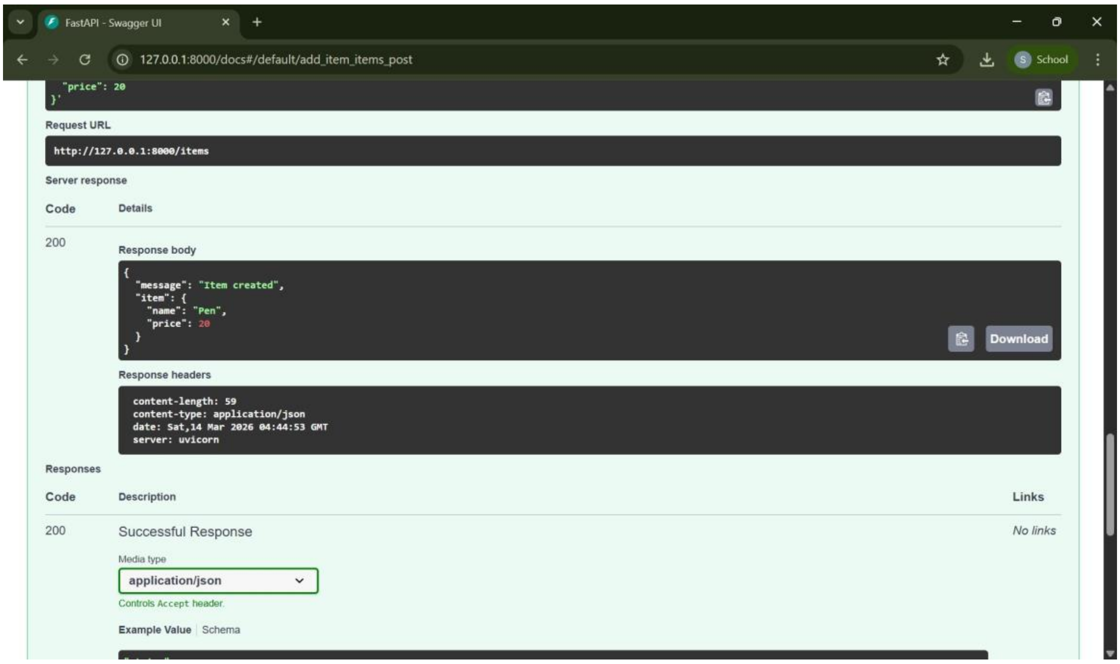
```
 15.2']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [23564] using StatReload
INFO:     Started server process [20464]
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [23564] using StatReload
INFO:     Started server process [20464]
INFO:     Started reloader process [23564] using StatReload
INFO:     Started server process [20464]
INFO:     Waiting for application startup.
INFO:     Started server process [20464]
INFO:     Waiting for application startup.
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

GET

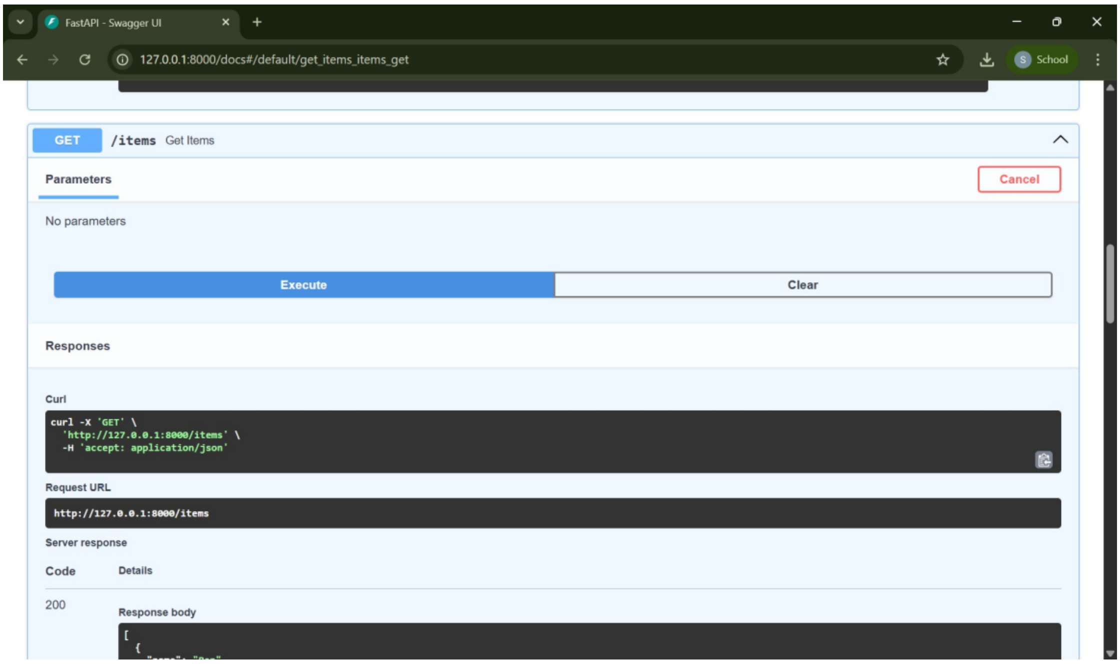

FastAPI - Swagger UI
127.0.0.1:8000/docs#/default/add_item_items_post
School
FastAPI 0.1.0 OAS 3.1
/openapi.json
default
GET / Read Root
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://127.0.0.1:8000/' \
-H 'accept: application/json'
Request URL
http://127.0.0.1:8000/

POST


FastAPI - Swagger UI
127.0.0.1:8000/docs#/default/add_item_items_post
School
POST /items Add Item
Parameters
Cancel
No parameters
Request body required
application/json
Edit Value | Schema
{
"additionalProp1": {}
}
Execute



RECHECK GET



Explanation

FastAPI. First of all, I created a list of items in Python to hold product information in the meantime. Then I inserted a GET /items endpoint to retrieve all the items and present them. I then added a POST / items endpoint where the user is allowed to add a new item and he/she sends data like name and price. Once all the code had been developed I started the FastAPI server and tested it with the Swagger documentation page (/docs). The GET request showed an empty list at first, but it could add an item with details and then it was displayed in the list after the POST request was sent.

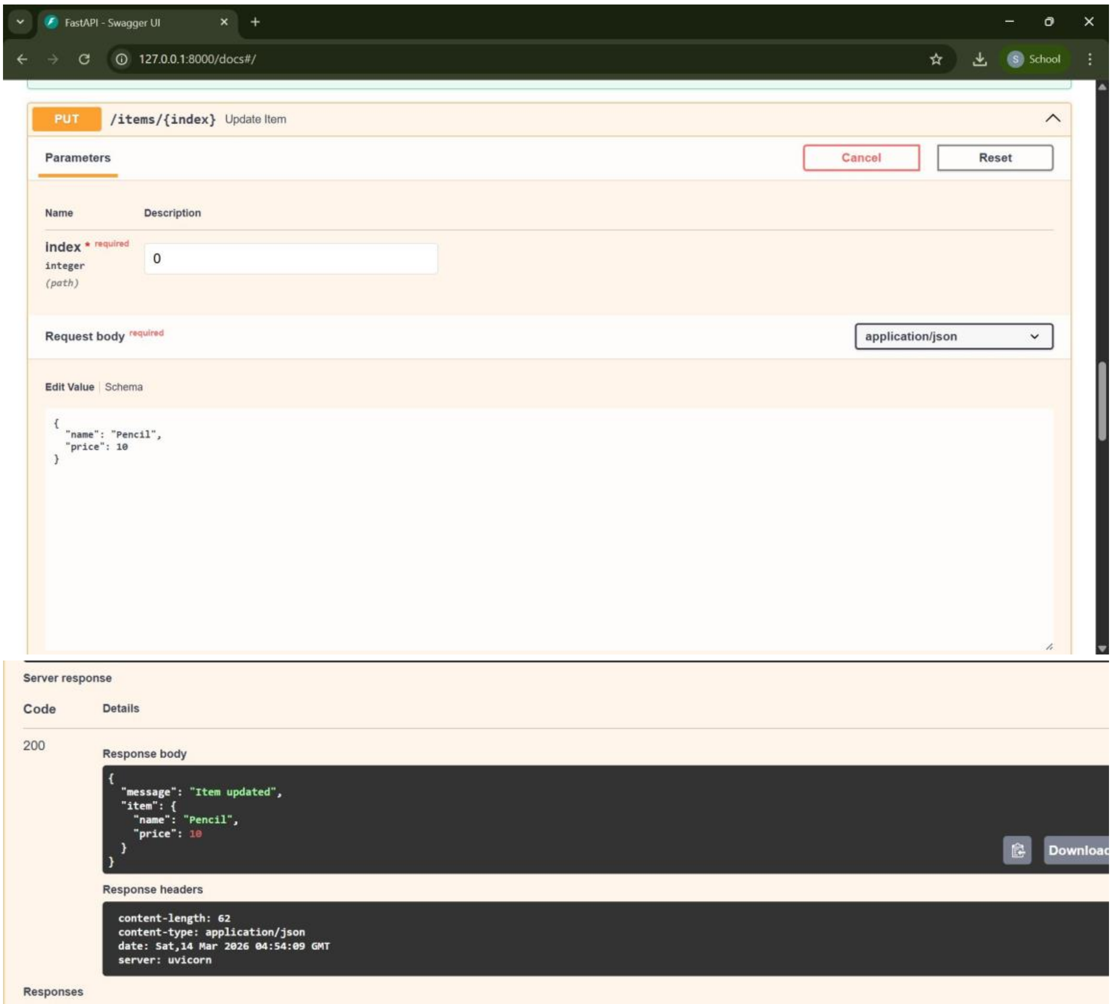
Task Descriptor- 3: (Update Existing Item)

Code

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
from fastapi import FastAPI

app = FastAPI()

# Temporary storage
items = []

# Root endpoint
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}

# Read items
@app.get("/items")
def get_items():
    return items
```

```python
# Create item
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}

@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
```

Output



Explanation

In this assignment, I conducted update operation of the existing items in Fastapi application. In order to update the items, I created a PUT endpoint, / items /index/ which will allow updating an item based on its rank in the items list (index). At this point, the node checks to see whether the index in question is in the list or not. In the case whereby the index is valid, then the details of the new item provided in the request overrides the current item. After the execution of the server, I tested the endpoint with the help of (/docs) and was able to update the information about the item.

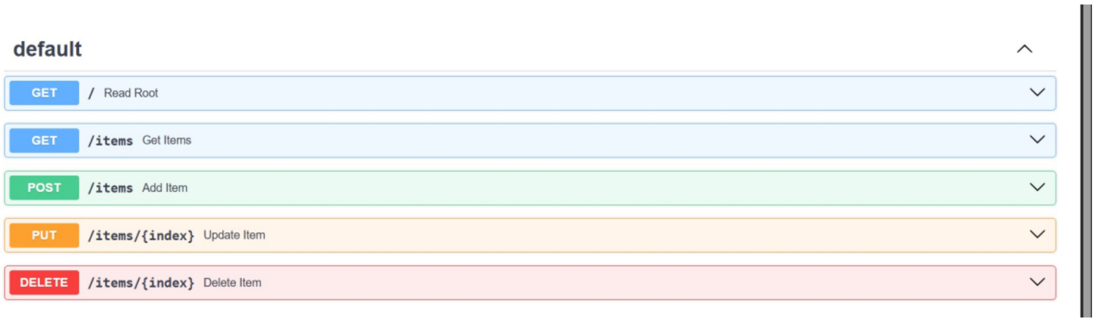
Task Descriptor- 4: (Delete Item)

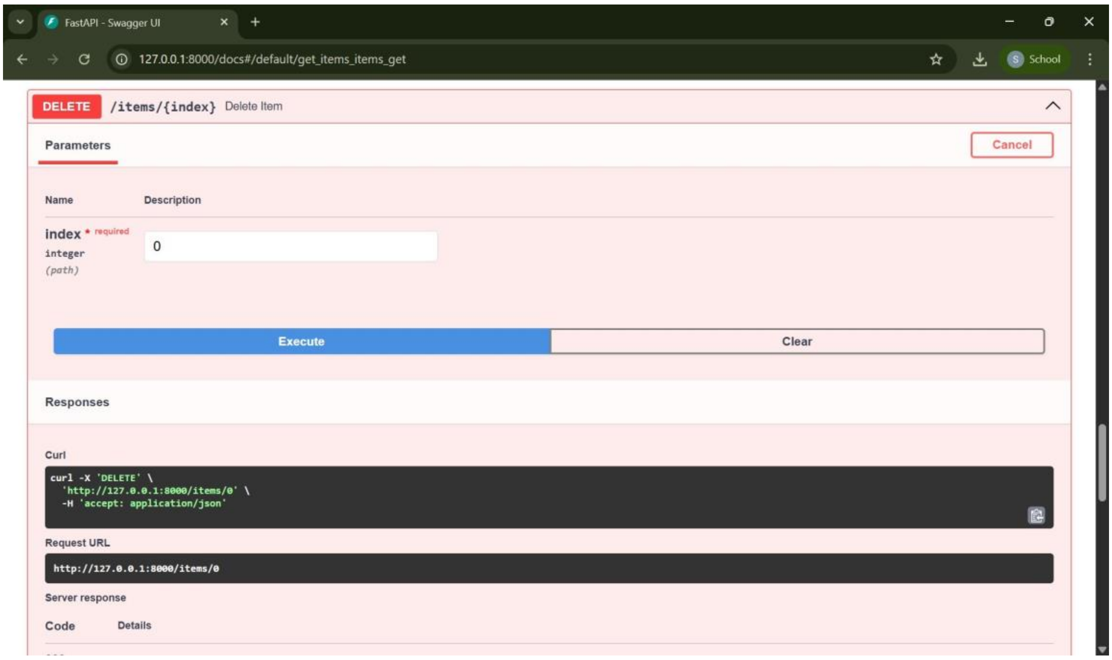
Code

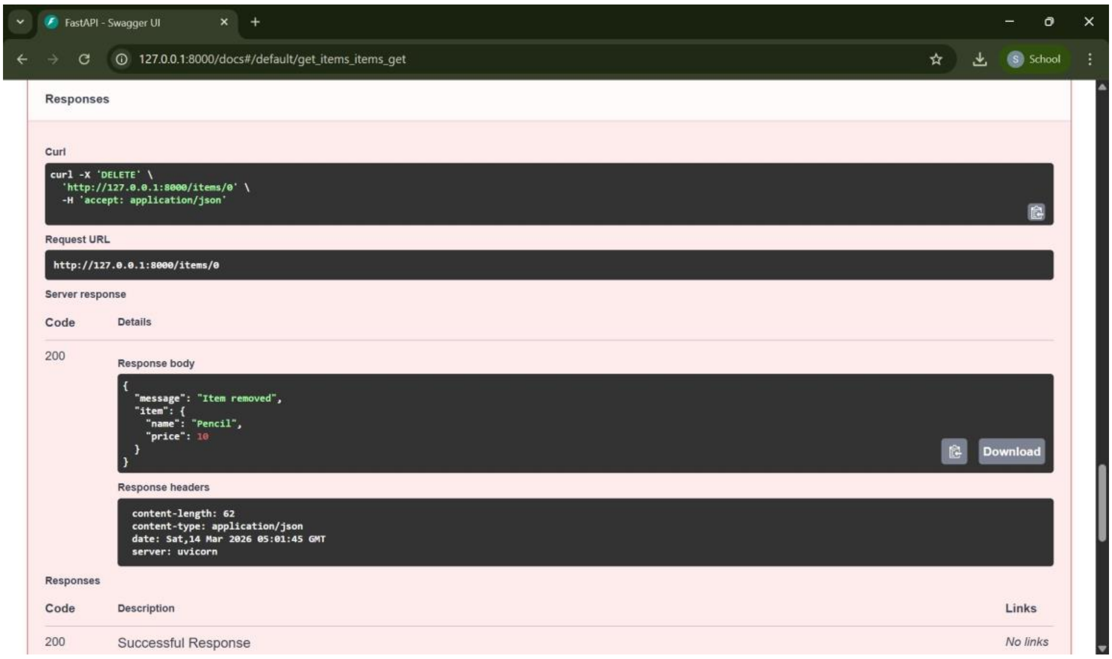
```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
from fastapi import FastAPI

app = FastAPI()

# Temporary storage
items = []

# Root endpoint
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}

# Read items
@app.get("/items")
def get_items():
    return items
```

```python
# Create item
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}

@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}

@app.delete("/items/{index}")
def delete_item(index: int):
    if index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

Output


default
GET / Read Root
GET /items Get Items
POST /items Add Item
PUT /items/{index} Update Item
DELETE /items/{index} Delete Item


FastAPI - Swagger UI
127.0.0.1:8000/docs#/default/get_items_items_get
School
DELETE /items/{index} Delete Item
Parameters
Cancel
Name
Description
index * required
integer
(path)
0
Execute
Clear
Responses
Curl
curl -X 'DELETE' \
'http://127.0.0.1:8000/items/0' \
-H 'accept: application/json'
Request URL
http://127.0.0.1:8000/items/0
Server response
Code
Details


FastAPI - Swagger UI
127.0.0.1:8000/docs#/default/get_items_items_get
School
Responses
Curl
curl -X 'DELETE' \
'http://127.0.0.1:8000/items/0' \
-H 'accept: application/json'
Request URL
http://127.0.0.1:8000/items/0
Server response
Code
Details
200
Response body
{
"message": "Item removed",
"item": {
"name": "Pencil",
"price": 10
}
}
Download
Response headers
content-length: 62
content-type: application/json
date: Sat,14 Mar 2026 05:01:45 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links

Explanation

In this assignment, I used a delete operation of the items in FastAPI application. I developed a delete endpoint, /items/ deleteEndpoint/index/ that removes an item in the list using its index. The first endpoint will examine whether the index provided is within the items list or not. In case the index is a valid one, then the item is taken out of the list and the deleted item details are returned as a response. I used the Swagger documentation page to test the endpoint and ensured that the item I chose was deleted successfully.

Task Descriptor -5: (Add Auto-Generated Documentation)

Prompt

Add simple Python docstrings and inline comments to this FastAPI code explaining what each endpoint does, its HTTP method, input parameters, and returned response.

Code

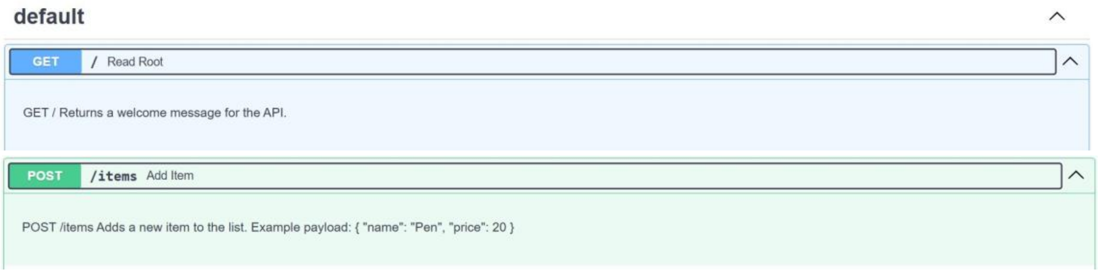
```python
#Add simple Python docstrings and inline comments to this FastAPI code explaining what each endpoint does, its HTT
from fastapi import FastAPI
app = FastAPI()
# Temporary storage for items
items = []
@app.get("/")
def read_root():
    """
    GET /
    Returns a welcome message to the user.
    """
    return {"message": "Welcome to AI-powered FastAPI"}
@app.get("/items")
def get_items():
    """
    GET /items
    Returns a list of all items in the store.
    """
    return items
@app.post("/items")
def add_item(item: dict):
    """
    POST /items
    Adds a new item to the store.

    Parameters:
    - item (dict): The item to be added, expected in JSON format.

    Returns:
    - A message confirming the creation of the item and the item itself.
    """
    items.append(item)
    return {"message": "Item created", "item": item}
```

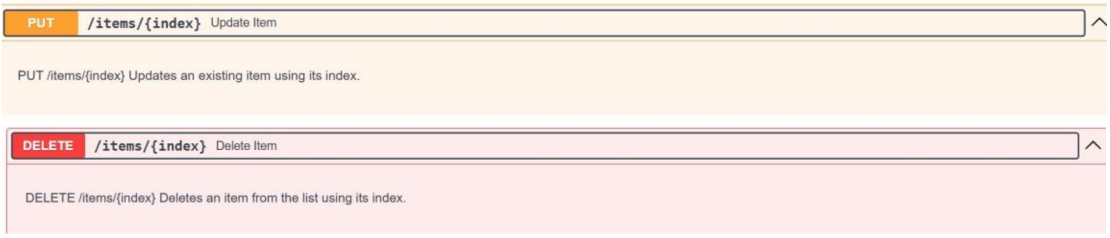
```python
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    """
    PUT /items/{index}
    Updates an existing item at the specified index.

    Parameters:
    - index (int): The index of the item to be updated.
    - item (dict): The new item data, expected in JSON format.

    Returns:
    - A message confirming the update of the item and the updated item itself.
    - An error message if the item is not found at the specified index.
    """
    if index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
@app.delete("/items/{index}")
def delete_item(index: int):
    """
    DELETE /items/{index}
    Deletes an existing item at the specified index.

    Parameters:
    - index (int): The index of the item to be deleted.

    Returns:
    - A message confirming the removal of the item and the removed item itself.
    - An error message if the item is not found at the specified index.
    """
    if index >= len(items):
        return {"error": "Item not found"}
```

```python
    """
    if index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

Output

```
INFO:     Will watch for changes in these directories: ['C:\\Users\\gauta\
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [9784] using StatReload
INFO:     Started server process [18872]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```





Explanation

Overall, in this task, I have included inline comments and docstrings on every API endpoint throughout the FastAPI application. These remarks outline the objective of every endpoint, the method of the HTTP request, and the anticipated input information. FastAPI automatically generates this API documentation with Swagger UI with the help of these docstrings. Once the server had been run, I accessed /docs and was presented with the auto generated documentation where all endpoints, request methods and example payloads were easily visible. This aids developers to learn and test the API with ease.